

Programming SQL Server 2019

User-Defined Functions

October 25, 2019 12:40:28 PM

Prelude

You have already learnt about VIEWS and Stored Procedures. In this lesson we going learn another very important object – User-Defined Function (UDF). UDFs are like stored procedures but differences are in how data is returned and how they are used.

Go ahead and get started.

Today's topics

- What are UDFs
- UDF types
- Stored Procedures Vs. UDFs
- Advantages of UDFs
- Creating Scalar UDFs
- Creating Inline Table-valued UDF
- Creating Multi-Statement Table-valued UDF
- Workouts
- Practice works

Home works

- A. Create a UDF that current date stripping off the time part.
- B. Create a UDF that returns a random number in between 1 and 10
- C. Create a UDF that accepts CustomerID and returns total order amount of the customer from Customers table in Northwind sample database
- D. Create a UDF that returns total order amount of a particular year.

Important Points:

-
-
-
-

Summary:

-
-
-
-

What is a User Defined Function (UDF)?

- User-defined functions are routines that accept parameters, perform an action, such as a complex calculation, and return the result of that action as a value.
- The return value can either be a single scalar value or a result set.

Stored Procedures vs. UDFs

- A user-defined function is, much like a stored proc, an ordered set of T-SQL statements that are pre-optimized and compiled and can be called to work as a single unit.
- The primary difference between them is how results are returned.
- A stored procedure can return only integer value but a UDF can return value of any data type even table
- You can pass parameter to a UDF but there is no OUTPUT parameter
- Unlike Stored Procedures, A UDF can be used in column-list or after FROM clause in SELECT query.

Types of UDF

- Primarily two types
- Scalar: User-defined scalar functions return a single data value
- Table-valued: User-defined table-valued functions return a table data type, i.e., result set. These are often called view with parameters
- Table-valued UDF can be of two types
 - Inline Table-Valued UDF
An Inline Table-Value user-defined function returns a table data type. It contains only one SELECT statement.
 - Multi-statement Table-Valued UDF
A Multi-Statement Table-Value user-defined function returns a table and can contain multiple SELECT statements to build the result.

Advantages of UDFs

- Scalar - these types of UDF can be used wherever an expression or variable can be used. These are useful when mathematical calculations or utility jobs are required.
- Table-valued - since these return table and can be used where tables or views are used. UDFs have some advantage over views that UDFs can accept parameter but views cannot.

Creating Scalar UDF

- Scalar functions return a single data value of the type defined in a RETURNS clause.
- All scalar data types, including bigint and sql_variant, can be used.
- The body of the function, defined in a BEGIN...END block, contains the series of Transact-SQL statements that return the value
- Syntax

```
CREATE FUNCTION function-name (parameter1,
parameter2, ...)
RETURNS data-type
AS
BEGIN
    SQL statement
    ...
    RETURN ...
END
GO
```
- parameter definition includes parameter name, data type and optionally default value
@parameter-name data-type = default-value

Workouts

- To experiment with UDFs, Let's create a table and add some sample data to it.
- Create the table
CREATE TABLE HomeSales
(

```
[date] DATETIME DEFAULT GETDATE (),
itemsold VARCHAR(30) NOT NULL,
unitprice MONEY NOT NULL,
quantity INT NOT NULL,
discountpercent FLOAT DEFAULT .05
```

```
)
GO
```

Add sample data data tape and optionally default value

```
INSERT INTO HomeSales VALUES
('2012-12-1', 'Rice Pack', 45.00, 3, .05),
('2012-12-1', 'Dal Pack', 80.00, 5, .10),
('2012-12-1', 'Wash Powder', 78.00, 5, .10),
('2012-12-2', 'Noodles', 33.00, 10, .075),
('2012-12-2', 'Baking Soda', 12.00, 9, .10),
('2012-12-3', 'Tea Bags', 98.00, 11, .15),
('2012-12-3', 'V. Oil', 280.00, 3, .12)
```

```
GO
```

date	product	unitprice	quantity	discount
1-12-12	Rice Pack	45.00	3	.05
1-12-12	Dal Pack	80.00	5	.10
1-12-12	Was powder	78.00	5	.10
2-12-12	Noodles	33.00	10	.075
2-12-12	Baking Soda	12.00	9	.10
3-12-12	Tea Bags	98.00	11	.15
3-12-12	V. Oil	280.00	3	.12

Workout #01

- Let's create a function that will calculate discount on sales
- The function will be passed unitprice, quantity sold and discount rate
- The function is

```
CREATE FUNCTION fnCalcDiscount
(@unitprice MONEY, @quantitysold INT,
@discountpercent FLOAT)
RETURNS MONEY
AS
BEGIN
    DECLARE @discountearned MONEY
    SET @discountearned =
        @unitprice*@quantitysold*@ discountpercent
    RETURN @discountearned
```

```
END
```

```
GO
```

- Let's use the UDF we just created
- You can use UDFs where an expression can be used
- To refer a scalar UDF, you have to use two-part name: schema-name.function-name
- Try this
SELECT dbo. fnCalcDiscount(70.00, 10, .10) 'Discount'
- You will get the result like below

Discount
70.00

- But the following will produce error
SELECT fnCalcDiscount (70.00, 10, .10) 'Discount'
- Let's try more
SELECT *, dbo.fnCalcDiscount (unitprice, quantity, discountpercent) 'save'

```
FROM HomeSales
GO
```

Result

date	product	unitprice	quantity	discount	save
1-12-12	Rice Pack	45.00	3	.05	6.75
1-12-12	Dal Pack	80.00	5	.10	20.00
1-12-12	Was powder	78.00	5	.10	39.00
2-12-12	Noodles	33.00	10	.075	24.75
2-12-12	Baking Soda	12.00	9	.10	10.80
3-12-12	Tea Bags	98.00	11	.15	161.70
3-12-12	V. Oil	280.00	3	.12	100.80

Workout #02

- Let's create another scalar function
- The function returns total sales amount, in a particular month, by summing up sales value (unitprice * quantity - discount amount)
- It takes year and month as arguments.
- The function


```
CREATE FUNCTION fnCalcDiscount
(@year INT, @month INT)
RETURNS MONEY
AS
BEGIN
    DECLARE @totalsales MONEY
    SELECT @totalsales = unitprice *
                quantity*(1-discountpercent)
    FROM Homesales
    WHERE YEAR([date])=@year AND
           MONTH([date]) = @month
    RETURN @totalsales
END
GO
```
- Test the function


```
SELECT dbo.fnCalcDiscount(2012, 12) 'Sales Dec 2012'
```
- Result

Sales Dec 2012
2897.20

Inline Table-valued Function

- An inline table-valued function, there is no function body; the table is the result set of a single SELECT statement
- The inline table-valued functions have one select statement after return block.
- There no BEGIN..END block
- Syntax


```
CREATE FUNCTION functionname (parameter1,
parameter2, ...)
RETURNS TABLE
AS
RETURN (
    SELECT statement
)
GO
```

Workout #03

- Let's create a inline table-valued function to find sales summary of a month


```
CREATE FUNCTION fnSalesSummaryOfMonth (
@year INT, @month INT)
RETURNS TABLE
AS
RETURN (
    SELECT
    SUM(unitprice * quantity) 'Total Sale',
    SUM(unitprice*quantity*discountpercent) 'Total Discount',
    SUM (unitprice * quantity*(1-discountpercent)) 'Net Amount'
```

```
FROM Homesales
WHERE YEAR([date])=@year AND
      MONTH([date]) = @month
)
GO
```

- Inline functions can be used to achieve the functionality of parameterized views
- Inline user-defined functions follow these rules:
 - The RETURNS clause contains only the keyword table
 - There is no function body delimited by BEGIN and END
 - The RETURN clause contains a single SELECT statement in parentheses
 - The table-valued function accepts only constants or @local_variable arguments.

- Test the function


```
SELECT * FROM fnSalesSummaryOfMonth (2012, 12)
GO
```
- Result

Total Sale	Total Discount	Net Amount
3281.00	383.8	2897.2

Multi-statement Table-valued Function

- In a multi-statement table-valued function, the function body, defined in a BEGIN...END block, contains the TRANSACT-SQL statements that build and insert rows into the table that will be returned.
- The multi-statement table-valued function has the following:
 - The RETURNS clause defines a local return variable name for the table returned by the function. The RETURNS clause also defines the format of the table.
 - The Transact-SQL statements in the function body build and insert rows into the return variable defined by the RETURNS clause.
 - When a RETURN statement is executed, the rows inserted into the variable are returned as the tabular output of the function. The RETURN statement cannot have an argument.
- Syntax


```
CREATE FUNCTION function-name (parameter1,
parameter2 ...)
RETURNS @table-variable TABLE (table-definition)
AS
BEGIN
    statements
    RETURN
END
GO
```

Workout #04

- Let's create a multi-statement table-valued function
- The function returns sales details of a particular month
- It takes year and month as arguments.
- The Function


```
CREATE FUNCTION fnSalesDetails (@year INT, @month INT)
RETURNS @salesdetails TABLE
(
    itemsold VARCHAR(30),
    totalsale MONEY,
    discount MONEY,
    net MONEY
)
AS
BEGIN
    INSERT INTO @salesdetails
    SELECT itemsold,
```

```

SUM(unitprice * quantity),
SUM(unitprice*quantity*discountpercent),
SUM(unitprice * quantity*(1-discountpercent))
FROM HomeSales
WHERE YEAR([date])=@year AND
      MONTH([date]) = @month
GROUP BY itemsold

```

RETURN

END

GO

- Test the function
SELECT * FROM fnSalesDetails(2012, 12)
GO
- Result

itemsold	totalsale	discount	net
Baking Soda	108.00	10.80	97.20
Dal Pack	400.00	40.00	360.00
Noodles	330.00	24.75	305.25
Rice Pack	135.00	6.75	128.25
Tea Bags	1078.00	161.70	916.30
V. Oil	840.00	100.80	739.20
Wash Powder	390.00	39.00	351.00

Practice #01

- A. Create a table "dotnettutorial" as described

Column name	Data type
topic	VARCHAR(30)
rating	float

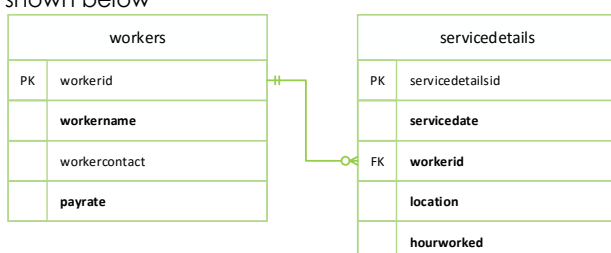
- B. Add following data into the table:

topic	rating
Static data	4.5
Extension methods	3.75
Interfaces	4.25
Delegates and Events	5.0
Lambda expressions	3.0
LINQ to objects	4.75
Threading	2.0
Generics	5.0

- C. Create a UDF that returns average rating
D. Create a UDF that returns tutorial topics of highest ratings
E. Create a table-valued UDF that returns highest rating, lowest rating and average rating
F. Convert the UDF created in E into multi-statement table-valued UDF
G. Create a UDF that accepts a float value and returns the topics that carry rating equal or greater than the value. Include topic and rating columns.

Practice #02

A home service provider company has some workers to serve customers at home. The Company wants to use a database to store service details. Two tables are designed as shown below



- A. Create the two tables with relation and constraints
B. Insert following data in workers table

workerid	workername	workercontact	payrate
1	Hard Worker	019999999	300.00
2	Jack of all trades		275.00
3	Lazy Joe	019999999	250.00
4	Sleeping Tom		200.00

Insert the following in servicedetails table

The day before Yesterday the flowing service has been provided

workerid	location	hourworked
1	Dhanmondi	3.5
4	Mirpur	2.5
2	Firmgate	1.0
2	Firmgate	4.0
1	Mirpur	6.5

Yesterday the flowing service has been provided

workerid	location	hourworked
2	Dhanmondi	3.5
1	Mirpur	2.5
2	Firmgate	1.0
3	Firmgate	4.0
4	Mirpur	6.5

- C. Create a view that shows workerid, workername along with total hourworked, and total pay in the current month.
[Pay is calculated using formula payrate*hourworked]
D. Create a view that shows workerid, workername along with total hourworked, and total pay in the last 7 days.
E. Create a view that shows workerid, workername along with his services including servicedate, location and hourworked.
F. Create a UDF that returns worker information with total hourworked and total pay within a date range
G. Create a UDF that returns location-wise total hourworked